TECHNICAL UNIVERSITY OF MOLDOVA
Faculty of Computers, Informatics, and Microelectronics
Department of Software Engineering
Software Engineering Study Programme

LABORATORY REPORT

Lab 4: The xv6 Shell from the Outside

Operating Systems

Student: Gafenco Victor

Group: FAF-241

Instructor: Assistant lecturer, Patricia Reițman

Chișinău - 2026

CONTENTS

Abstract **3**

Introduction **4**

1 Adapting and testing the sleep utility **5**

1.1 Interface and build integration 5

1.2 Execution results and limits 5

2 Comparing the two shells **7**

2.1 Method and results 7

2.2 Directory changes and executable search 8

2.3 Shared mechanisms and missing conveniences 9

3 Inspecting the argument vector **10**

3.1 Two implementations of the same observation 10

3.2 The wildcard and quoted-space experiment 10

3.3 Other argument-boundary tests 11

4 Explaining the results from the source **12**

4.1 The five command kinds 12

4.2 Why cd belongs to the parent shell 13

4.3 The absence of expansion code 13

Conclusions **15**

ABSTRACT

This laboratory compares a production command interpreter with the small shell of xv6. A pinned xv6 revision was built on openSUSE Tumbleweed using a RISC-V cross-compiler and booted in QEMU. A sleep utility and an argument-inspection program were registered and tested. Parallel Bash and xv6 experiments establish which facilities are shared and which require additional shell interpretation. Reading the source explains the absence of expansion, the treatment of quotes, and a limitation of directory changes inside compound command lines.

Keywords: xv6, Bash, RISC-V, exec, argument vector, shell parsing

INTRODUCTION

The objective is to distinguish kernel mechanisms from shell conveniences by observing the same command text under two interpreters. The handout assigns four parts: sleep compatibility, a shell comparison, inspection of argument vectors, and source analysis [**?**].

The host is openSUSE Tumbleweed. Revision `06aad25c735fd3159bdfae5680be4eab7b1668b2` of xv6 is checked out on branch `lab` in `~/xv6-riscv`. GCC 14.4.0 with the `riscv64-suse-linux-` prefix and QEMU 11.1.2 provide the build and execution environment. The course preparation check built the kernel and filesystem and verified a successful boot and directory listing. Figures show saved terminal output; the corresponding plain text is retained with the laboratory files.

1 ADAPTING AND TESTING THE SLEEP UTILITY

1.1 Interface and build integration

The user header declares `int pause(int);` at line 25. The source retained from Lab 2 already calls this function, so the correct adaptation is to preserve that call while keeping the executable's name `sleep`. The program validates that exactly one operand is supplied, converts it with `atoi`, calls `pause`, and exits. Incorrect argument counts produce a diagnostic on standard error and terminate with status 1.

```
$ grep -n "pause\|sleep" user/user.h
25:int pause(int);
```

Figure 1.1 – The pause declaration used by this xv6 revision

The Makefile registers both `$U/_sleep` and `$U/_args` as tab-indented UPROGS entries with continuation backslashes. A blank line still terminates the continued list. An additional compiler-detection branch recognizes openSUSE's prefix, allowing the ordinary `make qemu` command to work without a manually exported variable. The complete Makefile and a focused patch accompany the report.

```
$ cat user/sleep.c
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if(argc != 2){
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }

  pause(atoi(argv[1]));
  exit(0);
}
$ grep -n _sleep Makefile
136:    $U/_sleep\
```

Figure 1.2 – Sleep source and its UPROGS registration

1.2 Execution results and limits

The command `sleep 20` returned normally to the xv6 prompt. Both `sleep` and `sleep 10 20` printed `usage: sleep <ticks>`. These observations establish executable availability and argument-count handling. They do not independently measure the exact waiting duration or display the process exit status. The implementation follows the exercise's numeric-conversion scope and does not fully validate malformed

tick strings.

```
$ sleep 20
$ sleep
usage: sleep <ticks>
$ sleep 10 20
usage: sleep <ticks>
```

Figure 1.3 – A valid call and two invalid argument counts

2 COMPARING THE TWO SHELLS

2.1 Method and results

Bash ran in `~/shtest`, with README and the Linux argument-program source available. xv6 ran from the root of its filesystem image, where the executables are installed. Both interpreters received the prescribed command lines. Differences in listing format are separated from differences in shell interpretation.

Input	Bash result	xv6 result
`echo hello > f ; cat < f`	hello	hello
`ls | grep READ`	README	README with type, inode and size columns
`(echo a; echo b) | wc`	2 lines, 2 words, 4 bytes	Same counts
`echo one ; echo two`	one and two on separate lines	Same
`echo bg &`	Job/PID notice and bg	bg; no Bash-style job notice
`ls *`	Expanded directory entries	`ls: cannot open *`
`echo $HOME`	/home/victor	Literal `$HOME`
`echo "a b"`	a b without quotes	`"a b"` including quotes
`mkdir d ; cd d ; ls ..`	Changes into d, lists parent	`exec cd failed`, then root listing
Up arrow, Tab, Ctrl+R	History, completion, reverse search	Raw input bytes; no equivalent editing action

```
LINUX$ cd ~/shtest
LINUX$ touch README
LINUX$ echo hello > f ; cat < f
hello
LINUX$ ls | grep READ
README
LINUX$ (echo a; echo b) | wc
      2       2       4
LINUX$ echo one ; echo two
one
two
LINUX$ echo bg &
[1] 657918
LINUX$ bg
wait
LINUX$ ls *
README  args_linux.c  f
LINUX$ echo $HOME
/home/victor
LINUX$ echo "a b"
a b
LINUX$ mkdir d ; cd d ; ls ..
README  args_linux.c  d  f
LINUX$ cd ~/shtest
```

Figure 2.1 – The comparison command sequence under Bash

```
$ echo hello > f ; cat < f
hello
$ ls | grep READ
README          2 2 2441
$ (echo a; echo b) | wc
2 2 4
$ echo one ; echo two
one
two
$ echo bg &
bg
$ ls *
ls: cannot open *
$ echo $HOME
$HOME
$ echo "a b"
"a b"
$ mkdir d ; cd d ; ls ..
exec cd failed
.               1 1 1024
..              1 1 1024
README          2 2 2441
args            2 3 34000
sleep           2 4 34008
cat             2 5 35368
echo            2 6 34280
forktest        2 7 17328
grep            2 8 42720
init            2 9 34736
kill            2 10 34216
ln              2 11 34024
ls              2 12 41496
mkdir           2 13 34256
rm              2 14 34240
sh              2 15 56128
stressfs        2 16 35128
usertests       2 17 200232
grind           2 18 50272
wc              2 19 36240
zombie          2 20 33624
logstress       2 21 36168
forphan         2 22 35072
dorphan         2 23 34536
sync            2 24 33688
console         3 25 0
f               2 26 6
d               1 27 32
```

Figure 2.2 – The same command sequence under xv6

2.2 Directory changes and executable search

The compound `mkdir d ; cd d ; ls ..` test reveals an important qualification to the handout. In this xv6 shell, `cd` is recognized only when it begins the whole input line before normal parsing. Embedded in a semicolon list, it is treated as an ordinary external executable and fails. Consequently the exact combined command never changes directory in xv6.

A subsequent standalone `cd d` succeeds. From that directory, `ls ..` fails with `exec ls failed`, while `/ls ..` works. There is no PATH search: the executable name `ls` is relative to the current directory, whereas `/ls` identifies the installed program in the root. The relative pathname `../ls` would also locate it from `/d`. The test ends with a standalone `cd /`.

```
$ cd d
$ ls ..
exec ls failed
$ /ls ..
.              1 1 1024
..             1 1 1024
README         2 2 2441
args           2 3 34000
sleep          2 4 34008
cat            2 5 35368
echo           2 6 34280
forktest       2 7 17328
grep           2 8 42720
init           2 9 34736
kill           2 10 34216
ln             2 11 34024
ls             2 12 41496
mkdir          2 13 34256
rm             2 14 34240
sh             2 15 56128
stressfs       2 16 35128
usertests      2 17 200232
grind          2 18 50272
wc             2 19 36240
zombie         2 20 33624
logstress      2 21 36168
forphan        2 22 35072
dorphan        2 23 34536
sync           2 24 33688
console        3 25 0
f              2 26 6
d              1 27 32
$ cd /
```

Figure 2.3 – Standalone directory change and the absolute-path workaround

2.3 Shared mechanisms and missing conveniences

Both shells provide process execution, input/output redirection, pipes, sequencing, grouping and background execution. Each can change its own current directory, although xv6's recognition is restricted. Bash additionally provides variable and pathname expansion, meaningful quote syntax, executable search, history, completion, conditional operators and job-control commands.

History and completion would be especially noticeable omissions in daily use. A fixed executable-directory fallback would be a relatively small first addition: after a suitable failed relative execution attempt, try a known program directory while retaining the semantics of explicit pathnames. Quote-aware parsing and general globbing would involve more extensive interpretation. This is an implementation assessment rather than a modification made in this laboratory.

3 INSPECTING THE ARGUMENT VECTOR

3.1 Two implementations of the same observation

The xv6 program includes its user headers, prints `argc`, prints each `argv[i]` inside brackets and exits through `exit(0)`. The Linux version uses `stdio.h` and returns zero. Neither version performs globbing, variable expansion or quote processing. The Linux build succeeds with `gcc -Wall -Wextra -Werror -o args args_linux.c`.

```
$ cat user/args.c
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  printf("argc = %d\n", argc);
  for(int i = 0; i < argc; i++)
    printf("argv[%d] = [%s]\n", i, argv[i]);
  exit(0);
}
$ grep -n _args Makefile
135:    $U/_args\
```

Figure 3.1 – Argument-inspection source and build registration

3.2 The wildcard and quoted-space experiment

In the Bash test directory, the matching C filename is `args_linux.c`. The two executions produce the following complete argument vectors:

```
LINUX$ ./args *.c "x y"
argc = 3
argv[0] = [./args]
argv[1] = [args_linux.c]
argv[2] = [x y]

$ args *.c "x y"
argc = 4
argv[0] = [args]
argv[1] = [*.c]
argv[2] = ["x]
argv[3] = [y"]
```

Bash passes two operands plus the program name, whereas xv6 passes three operands plus the program name. The number of Bash operands would increase if more C files matched. The quoted space survives inside one Bash argument, but xv6 separates at that space and retains the quote characters in the adjacent words.

3.3 Other argument-boundary tests

Operands typed	Bash	xv6
`one two three`	argc 4; three words	argc 4; same words
`$HOME`	argc 2; expanded home pathname	argc 2; literal dollar-name
`'single quoted'`	argc 2; one unquoted string	argc 3; two strings retaining quotes

```
$ args one             two         three
argc = 4
argv[0] = [args]
argv[1] = [one]
argv[2] = [two]
argv[3] = [three]
$ args *.c "x y"
argc = 4
argv[0] = [args]
argv[1] = [*.c]
argv[2] = ["x]
argv[3] = [y"]
$ args $HOME
argc = 2
argv[0] = [args]
argv[1] = [$HOME]
$ args 'single quoted'
argc = 3
argv[0] = [args]
argv[1] = ['single]
argv[2] = [quoted']
```

Figure 3.2 – All argument-vector tests in xv6

```
LINUX$ cd ~/shtest
LINUX$ gcc -Wall -Wextra -Werror -o args args_linux.c
LINUX$ ./args one              two         three
argc = 4
argv[0] = [./args]
argv[1] = [one]
argv[2] = [two]
argv[3] = [three]
LINUX$ ./args *.c "x y"
argc = 3
argv[0] = [./args]
argv[1] = [args_linux.c]
argv[2] = [x y]
LINUX$ ./args $HOME
argc = 2
argv[0] = [./args]
argv[1] = [/home/victor]
LINUX$ ./args 'single quoted'
argc = 2
argv[0] = [./args]
argv[1] = [single quoted]
```

Figure 3.3 – All argument-vector tests in Bash

Repeated unquoted whitespace separates words in both interpreters and does not create empty arguments. The shell constructs the vector before the executable begins. The kernel's execution interface transfers that vector to the new program; it does not implement Bash's wildcard or quoting language. The receiving program merely exposes the values that it was given.

4 EXPLAINING THE RESULTS FROM THE SOURCE

4.1 The five command kinds

The pinned `user/sh.c` contains 499 lines. Lines 8–12 define five node kinds:

EXEC An ordinary command word produces an execution node; no special operator is required. The execution case invokes `exec` with its argument vector.

REDIR The operators <, > and the parsed >> token produce redirection nodes. A descriptor is closed and reopened before the nested command runs.

PIPE The | operator produces a pipe node. Children connect their input and output using the pipe descriptors and `dup`.

LIST A semicolon produces a sequence node. The left side is forked and waited for before the right side runs.

BACK An ampersand produces a background node, which forks its nested command without waiting for that child in the node itself.

Parentheses invoke `parseblock` to group existing node kinds; they are not a sixth kind. Recognizing a token does not imply full Bash semantics: xv6's >> case opens with `O_WRONLY | O_CREATE` without an append flag or a seek, so it must not be assumed to append like Bash.

```
LINUX$ cd ~/xv6-riscv
LINUX$ git rev-parse HEAD
06aad25c735fd3159bdfae5680be4eab7b1668b2
LINUX$ wc -l user/sh.c
499 user/sh.c
LINUX$ grep -n "#define" user/sh.c
8:#define EXEC  1
9:#define REDIR 2
10:#define PIPE  3
11:#define LIST  4
12:#define BACK  5
14:#define MAXARGS 10
```

Figure 4.1 – Revision, source length and command-node definitions

```
LINUX$ grep -n "case " user/sh.c
75:  case EXEC:
83:  case REDIR:
93:  case LIST:
101:  case PIPE:
125:  case BACK:
284:  case 0:
286:  case '|':
287:  case '(':
288:  case ')':
289:  case ';':
290:  case '&':
291:  case '<':
294:  case '>':
391:    case '<':
394:    case '>':
397:    case '+': // >>
469:  case EXEC:
475:  case REDIR:
481:  case PIPE:
487:  case LIST:
493:  case BACK:
```

Figure 4.2 – Execution and parsing switch cases

4.2 Why cd belongs to the parent shell

Lines 166–170 handle `cd` directly in `main`, before the ordinary fork and parse path. The persistent shell process must call `chdir`; a separate child's directory change cannot update its parent. The position of this check also accounts for the failure of an embedded `cd` in the earlier list.

```
LINUX$ grep -n "cd" user/sh.c | head
170:        fprintf(2, "cannot cd %s\n", cmd + 3);
LINUX$ nl -ba user/sh.c | sed -n '156,179p'
   156      }
   157    }
   158
   159    // Read and run input commands.
   160    while (getcmd(buf, sizeof(buf)) >= 0) {
   161      char *cmd = buf;
   162      while (*cmd == ' ' || *cmd == '\t')
   163        cmd++;
   164      if (*cmd == '\n') // is a blank command
   165        continue;
   166      if (cmd[0] == 'c' && cmd[1] == 'd' && cmd[2] == ' ') {
   167        // Chdir must be called by the parent, not the child.
   168        cmd[strlen(cmd) - 1] = 0; // chop \n
   169        if (chdir(cmd + 3) < 0)
   170          fprintf(2, "cannot cd %s\n", cmd + 3);
   171      } else {
   172        if (fork1() == 0)
   173          runcmd(parsecmd(cmd));
   174        wait(0);
   175      }
   176    }
   177    exit(0);
   178  }
   179
```

Figure 4.3 – Directory changes handled before forking and parsing

4.3 The absence of expansion code

The token reader skips whitespace, recognizes operator characters and collects ordinary word spans. It has no state for single or double quotes, and no stage that expands variables or enumerates wildcard

matches. Searching for an asterisk finds C pointer syntax, not an implementation of globbing; searching for a dollar sign finds the prompt. Those search hits require interpretation in their surrounding code. This source reading agrees with the literal stars, dollar-names and quote characters measured by the argument program.

CONCLUSIONS

The completed build and boot establish a working xv6 environment on openSUSE. The sleep utility reuses the already-correct pause interface from Lab 2, and the argument utility isolates the shell's interpretation from application behaviour. Both shells rely on familiar process and descriptor mechanisms, but Bash performs substantial additional work before a program starts. The source analysis explains both the missing expansion features and the narrower treatment of directory changes. All prescribed command experiments were executed, and QEMU was shut down through its escape command so the filesystem image is released for subsequent use.